EMPLOYEE TURNOVER ANALYTICS

PREPARED BY

ARYAN VARSANI(112315205)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY PUNE

INTRODUCTION

- This project focuses on developing a machine learning solution to predict employee turnover by analyzing historical HR data. Key features such as the number of projects handled, average monthly working hours, time spent in the company, promotion history, salary level, and employee satisfaction are used to identify patterns related to attrition.

- The goal is to help the HR team by giving them smart predictions so they can make better decisions early, spend less on people leaving, and keep more employees for longer.

FEATURES

Column Name	Description
satisfaction_level	Satisfaction level at the job of an employee
last_evaluation	Rating between 0 and 1, received by an employee at his last evaluation
number_project	The number of projects an employee is involved in
average_montly_hours	Average number of hours in a month spent by an employee at the office
time_spend_company	Number of years spent in the company

FEATURES

Column Name	Description
promotion_last_5years	Number of promotions in his stay
Work_accident	0 - no accident during employee stay, 1 - accident during employee stay
Department	Department to which an employee belongs to
salary	Salary in USD
left	0 - an employee stays with the company, 1 - an employee left the company

PROJECT RATIONALE

01

Reduces Costs

When many employees leave a company, it becomes expensive to hire, train, and get new people ready for work. Predictive modeling can help stop this from happening and save money.

02

Improves Retention

Finding employees who might leave soon helps HR act quickly to keep them happy and make them stay longer.

03

Predictive HR Action

It helps the HR team get ready in advance instead of dealing with employee resignations after they happen.

04

Employee Experience

Knowing why employees are unhappy helps make better rules at work and keeps them more interested and involved.

TOOLS & LIBRARIES USED

01

Data Handling

- Numpy
- pandas

02

Data Visualization

- matplotlib
- seaborn

03

Data Preprocessing

- StandardScaler
- Label Encoding
- One-Hot Encoding
- Train-Test Split
- imputer
- SMOTE

04

Machine Learning

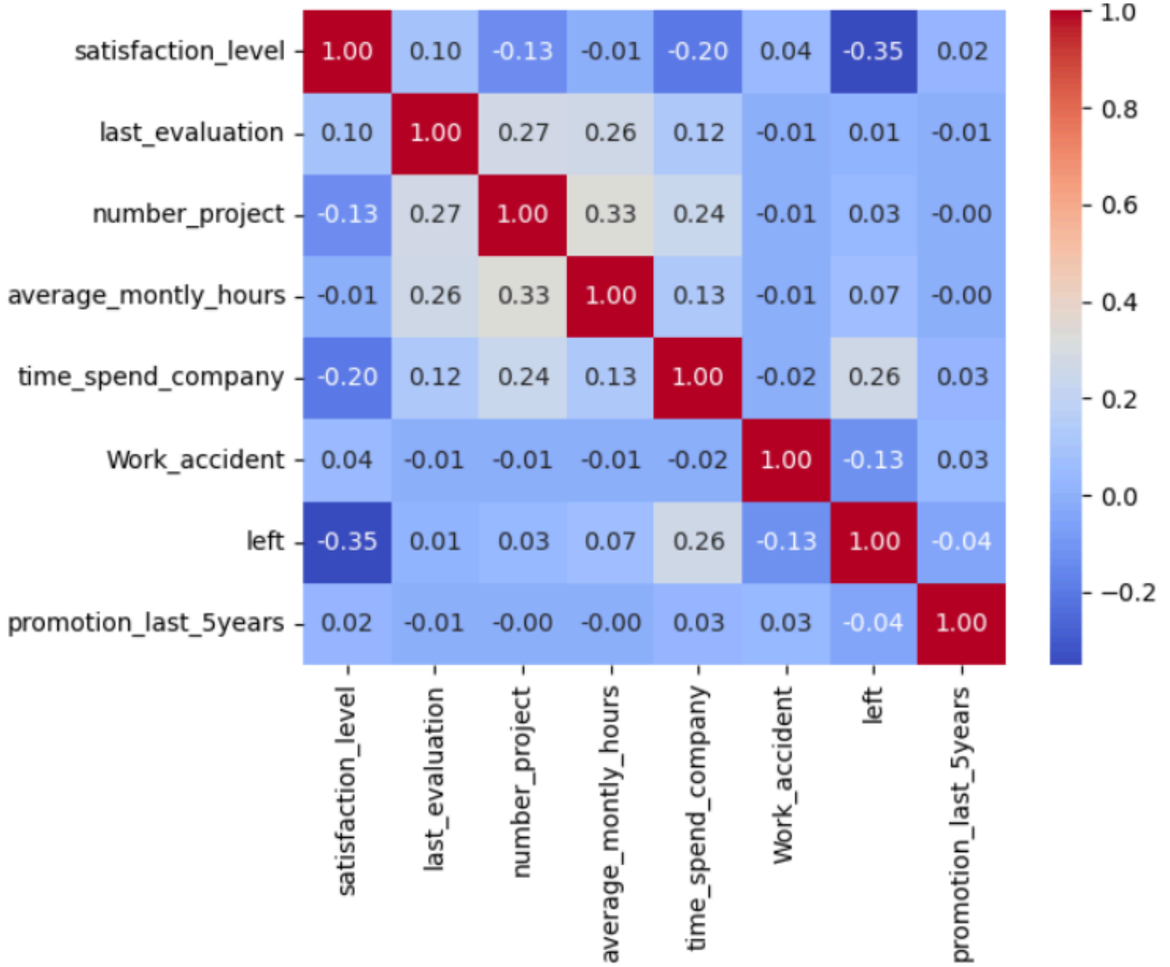
- Logistic Regression
- Random Forest
- Gradient Boosting
- K-Fold Cross-Validation
- Evaluation Metrics

05

Development

- python
- flask
- django
- Heroku
- Render
- AWS

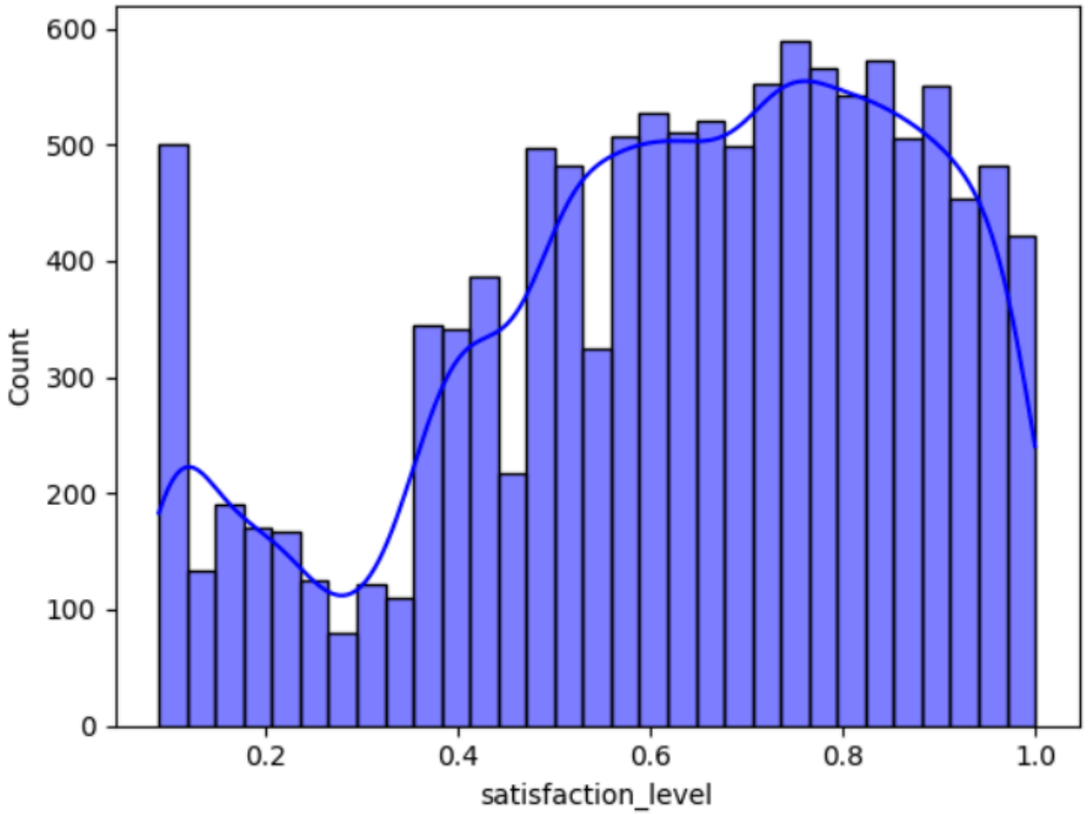
CORRELATION HEATMAP

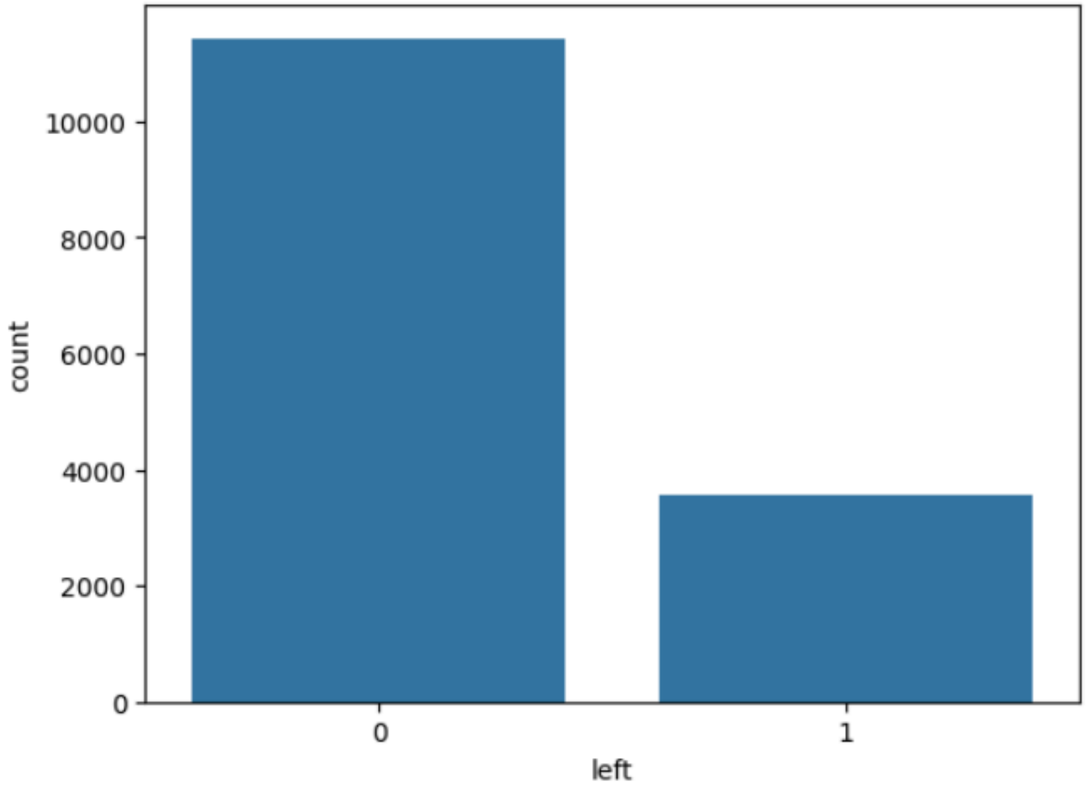


- The provided correlation matrix shows the relationships between various features and the target variable "left" (likely indicating employee turnover) in a binary classification dataset.
- Strong correlations are observed between "left" and "satisfaction_level" (-0.35), "time_spend_company" (0.26), and "promotion_last_5years" (-0.04), suggesting that lower satisfaction, longer tenure, and lack of promotion may influence turnover.
- This analysis shows that a model trying to predict if an employee will stay or leave should focus on these features the most during training.

DATA VISUALIZATION

"The graph shows that most employees have moderate to high satisfaction, with fewer showing very low satisfaction."





"As observed, the dataset is imbalanced, which may affect model performance. Therefore, applying SMOTE is recommended to balance the class distribution."

DATA PREPROCESSING

- **Handling Missing Values**

Checked for any null values and addressed them appropriately (e.g., removal or imputation).

- **Encoding Categorical Variables**

Converted non-numeric columns like department using Label Encoding / One-Hot Encoding.

- **Feature Scaling**

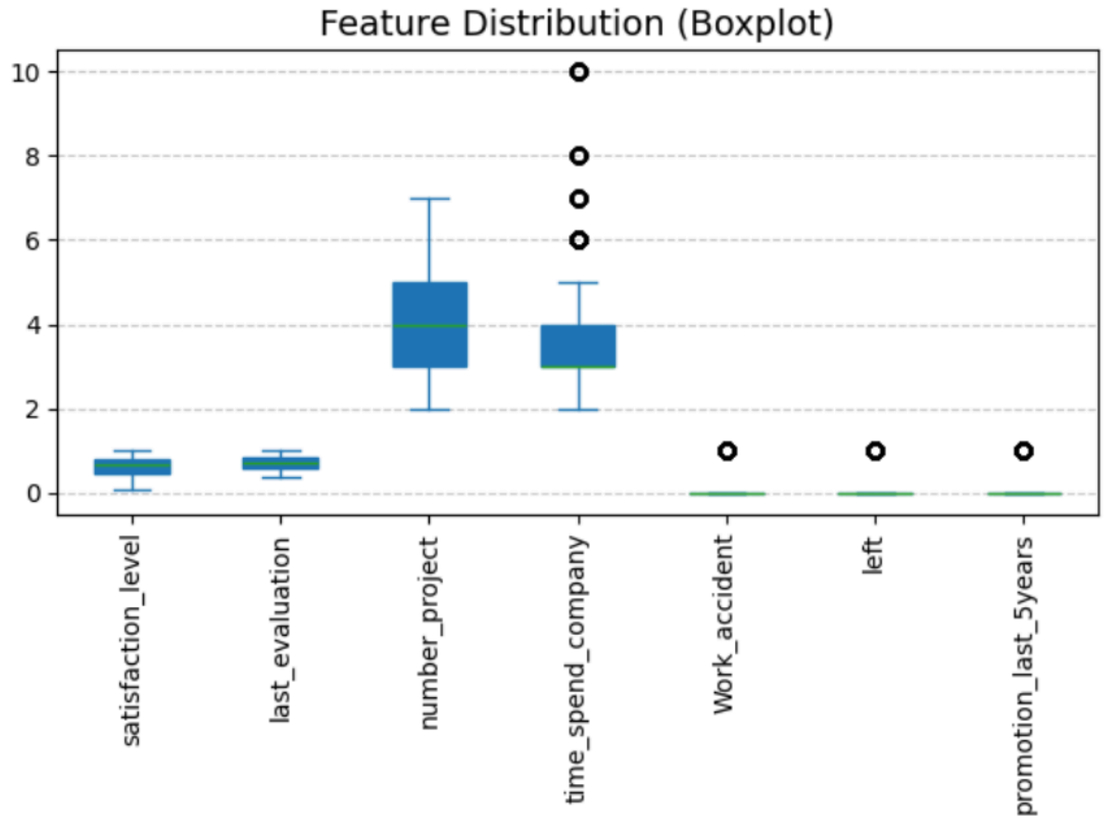
Applied normalization or standardization to numerical features for uniformity

- **Class Balancing**

Used SMOTE (Synthetic Minority Oversampling Technique) to address imbalance in the target variable (left column).

- **Outlier handling**

detecting and managing unusual data points that can distort analysis and affect model accuracy.



MODEL BUILDING

1. Train-Test Split

- Divided the dataset into 80% training and 20% testing data.

2. SMOTE (Synthetic Minority Oversampling Technique)

- Applied to the training set only to handle class imbalance.

3. Model Selection

- Trained multiple classifiers:
  - Logistic Regression
  - Random Forest
  - Gradient Boosting

4. Cross-Validation

- Used 5-fold cross-validation to validate model performance during training.

5. Hyperparameter Tuning

- Optimized models using GridSearchCV (or RandomizedSearchCV, if you used it).

6. Testing

- Evaluated the final model on the test set using performance metrics.

```
log_reg=LogisticRegression(max_iter=1000,random_state=42)
log_reg.fit(X_train_smote,y_train_smote)
```

```
LogisticRegression
LogisticRegression(max_iter=1000, random_state=42)
```

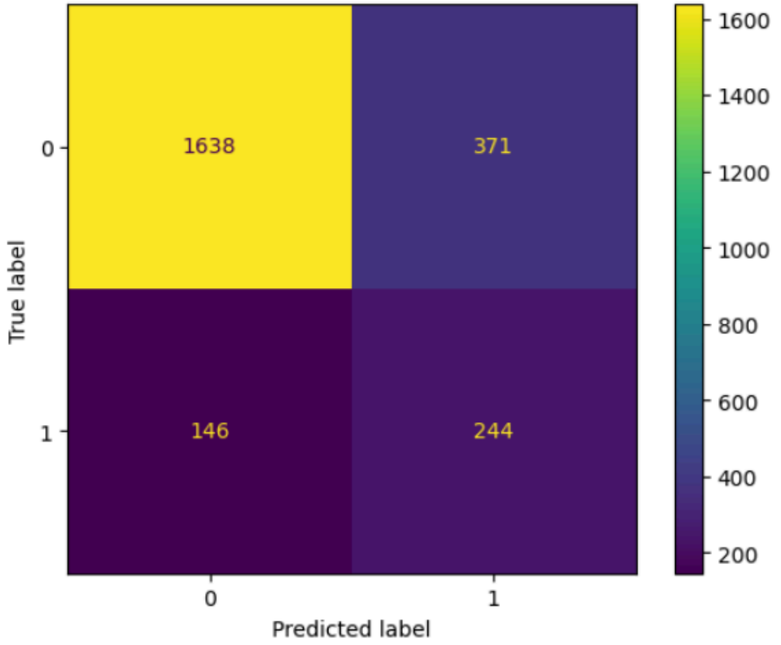
```
rf_model = RandomForestClassifier(random_state=123)
rf_model.fit(X_train_smote, y_train_smote)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=123)
```

```
gbc.fit(X_train_smote, y_train_smote)
```

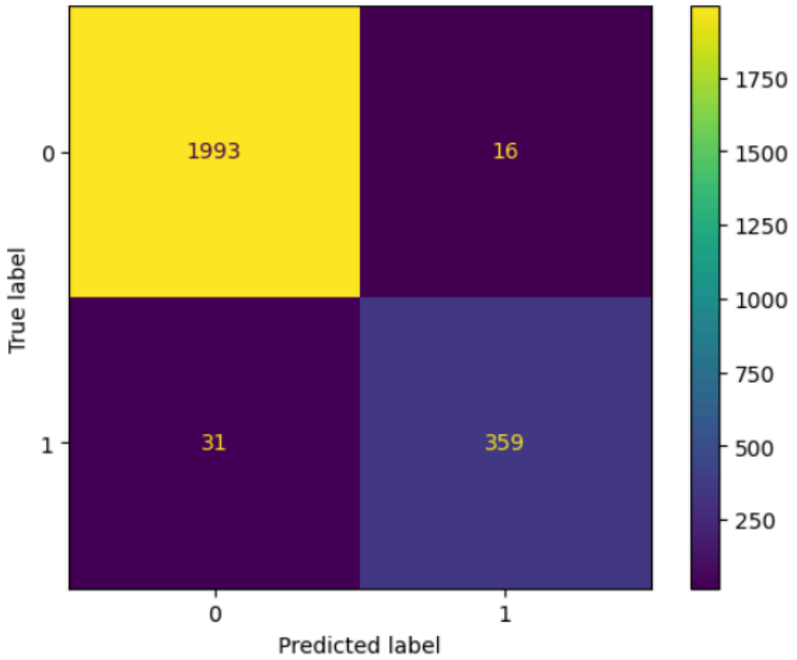
```
GradientBoostingClassifier
GradientBoostingClassifier(n_estimators=1000, random_state=7)
```

MODEL EVALUATION & RESULTS



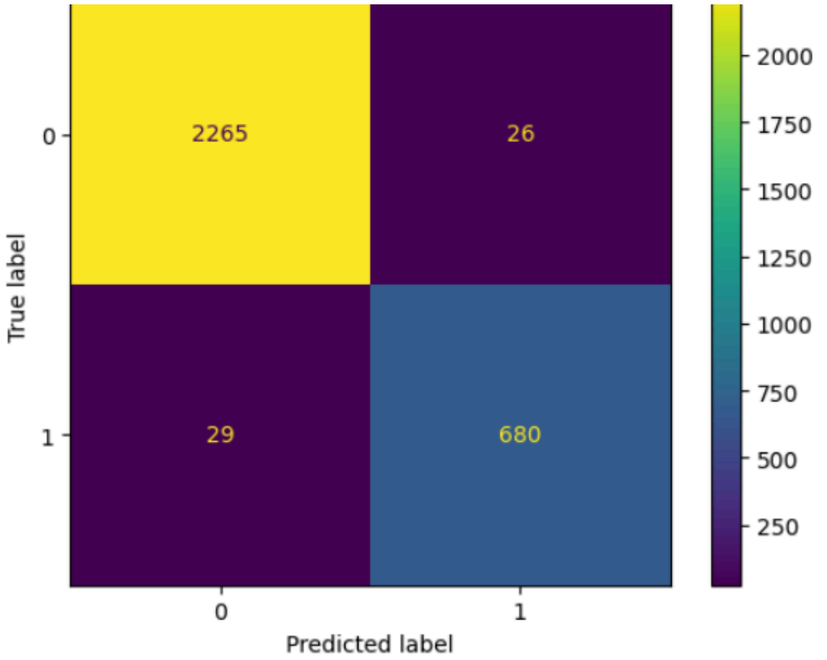
train accuracy : 86.83 %
test accuracy : 80.65 %

Logistic Regression



train accuracy : 100 %
test accuracy : 98.04 %

Random Forest

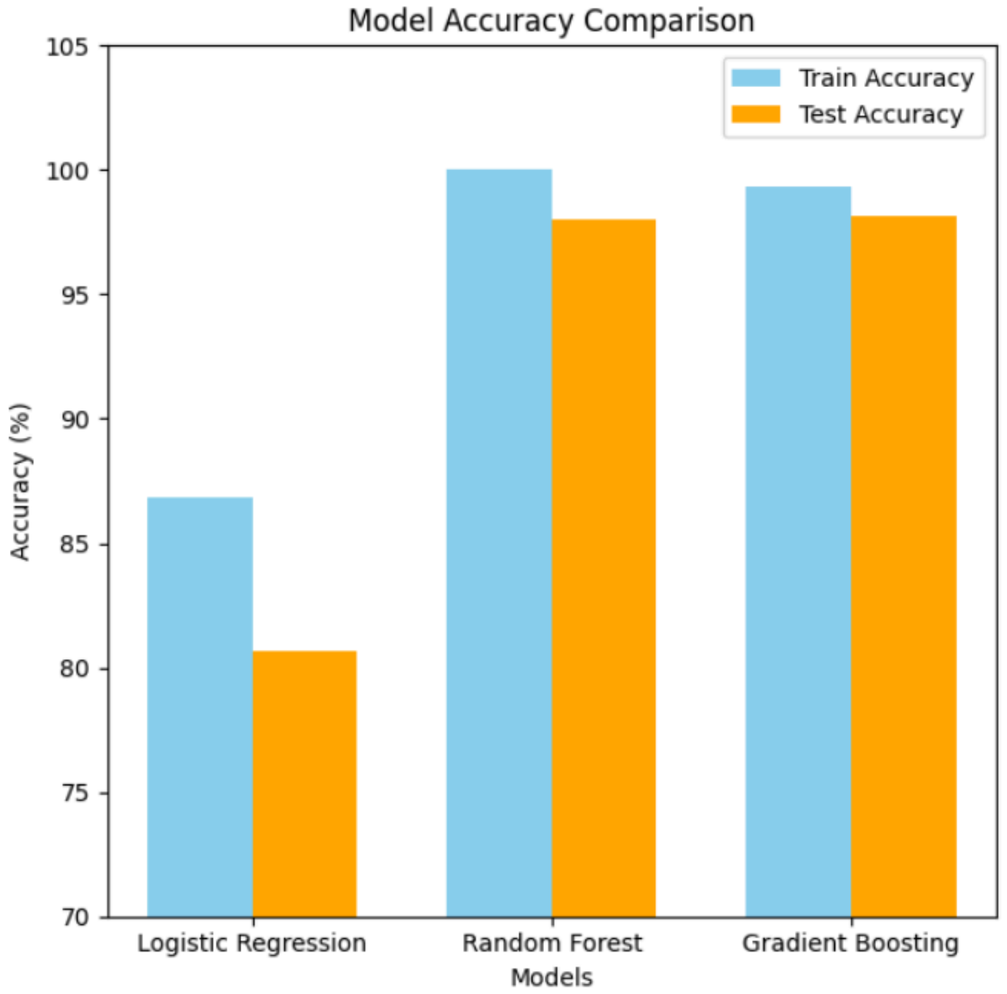


train accuracy : 99.32 %
test accuracy : 98.16 %

Gradient Boosting

CONCLUSION & KEY TAKEAWAYS

- Logistic Regression achieved 86.83% accuracy on the training set and 80.65% on the test set, indicating a slight overfitting tendency.
- Random Forest showed 100% training accuracy and 98.04% test accuracy, reflecting high performance with excellent generalization.
- Gradient Boosting reached 99.32% on training and 98.16% on testing, it is also excellent accuracy .
- Both Random Forest and Gradient Boosting achieved comparable high accuracy, significantly outperforming Logistic Regression.
- These results suggest that ensemble methods are better suited for capturing complex patterns in the data.



Thank you